

Aab o Hawa (آب و ہوا)

Karachi Air Quality Intelligence Platform

Project Report

Machine Learning Engineering

GitHub Repository:

https://github.com/Taq2005/Karachi_AQI

Live Application:

<https://karachiaqi-np4grgkcjpr4qxwpak3xep.streamlit.app/>

June 2026

1. Executive Summary

Aab o Hawa (آب و ہوا), meaning 'water and air' in Urdu, is an end-to-end machine learning platform for real-time air quality monitoring and forecasting in Karachi, Pakistan. Karachi consistently ranks among the most polluted cities in the world, yet no locally-built, data-driven tool existed to monitor and predict its air quality. This project addresses that gap.

The platform ingests hourly air quality and meteorological data from the Open-Meteo API, stores it in a MongoDB Atlas feature store, trains and evaluates four machine learning models daily via GitHub Actions, selects the best model by RMSE, and publishes a 72-hour recursive hourly forecast aggregated into 3-day daily predictions. The results are served through an interactive Streamlit dashboard accessible publicly at the URL above.

2. Problem Statement

Air pollution is a significant public health crisis in Karachi. The city's AQI frequently exceeds safe thresholds due to vehicular emissions, industrial activity, construction dust, and weather conditions that trap pollutants. Despite this, there exists no publicly accessible, real-time, locally-built forecasting tool tailored to Karachi.

This project aims to solve three problems:

- 1. Real-time monitoring: No consolidated dashboard exists for Karachi residents to check current AQI and individual pollutant levels.
- 2. Short-term forecasting: Residents, healthcare providers, and policymakers cannot plan activities around air quality without a 2-3 day forecast.
- 3. Automated pipeline: Any existing tools require manual updates; this project automates the full cycle from data ingestion to forecast publication.

3. System Architecture

The system is composed of four distinct layers that operate in a fully automated pipeline:

Layer	Component	Technology
Data Ingestion	Feature Pipeline	Open-Meteo APIs + MongoDB Atlas
ML Training	Training Pipeline	scikit-learn, LightGBM, XGBoost, GridFS
Automation	CI/CD	GitHub Actions (cron daily at 06:00 PKT)
Presentation	Dashboard	Streamlit + Plotly, deployed on Streamlit Cloud

4. Data Pipeline (feature_pipeline.py)

The feature pipeline is responsible for fetching, cleaning, feature-engineering, and storing hourly observations in MongoDB Atlas. It runs daily via GitHub Actions and is designed to be idempotent — safe to run multiple times without creating duplicate records.

4.1 Data Sources

- Open-Meteo Air Quality API: Provides hourly readings of PM2.5, PM10, NO2, Ozone, SO2, CO, and US AQI index for coordinates 24.8608°N, 67.0104°E (Karachi city centre).
- Open-Meteo Archive API: Provides historical hourly meteorological data including temperature, humidity, dew point, wind speed, wind direction, wind gusts, precipitation, surface pressure, cloud cover, and shortwave radiation.
- Open-Meteo Forecast API: Used specifically for today's data, since the archive API has a 1-2 day lag before the most recent data becomes available.

4.2 Smart Date Range Detection

Rather than fetching all data from scratch on each run, the pipeline uses a smart incremental approach. It first probes the archive API going back day by day to find the latest date for which complete data is available. It then queries MongoDB to find the timestamp of the most recently stored record. Only the gap between these two dates is fetched, with a 2-day overlap to catch any late-arriving data corrections.

```
last_doc = collection.find_one(sort=[("time", -1)])  
START_DATE = last_date - timedelta(days=2)    # 2-day overlap for safety
```

4.3 Dual-Source Today Fetch

Because the archive API lags by 1-2 days, today's real-time data is fetched separately from the Open-Meteo Forecast API. Only hours that have already elapsed are included, preventing future null values from being stored. The two sources are merged, deduplicated, and upserted into MongoDB using bulk_write with UpdateOne and upsert=True, ensuring no duplicates.

4.4 Feature Engineering

The following features are computed on the raw data before storage:

Feature	Description
hour, day_of_week, month	Calendar components extracted from timestamp
is_weekend	Binary flag: 1 if Saturday or Sunday
hour_sin, hour_cos	Cyclical encoding of hour ($\sin/\cos$ of $2\pi \times h/24$)
dow_sin, dow_cos	Cyclical encoding of day of week
pm_ratio	$\text{PM}_{2.5} / (\text{PM}_{10} + \text{epsilon})$ — relative fine particle fraction
aqi_lag_1h, aqi_lag_24h	AQI at 1 hour and 24 hours prior
aqi_roll_24h	24-hour rolling mean of AQI
aqi_change_rate	Hour-over-hour AQI difference

4.5 Data Storage

All processed records are stored in a MongoDB Atlas collection named 'hourly_features'. The upsert strategy ensures that re-running the pipeline never creates duplicate documents. The collection stores approximately 720 documents per month of data.

5. Training Pipeline (training_pipeline.py)

The training pipeline loads all historical data from MongoDB, performs additional hourly-level feature engineering, trains four ML models, evaluates them on a held-out test set, selects the best model, serialises it to MongoDB GridFS, and generates a 72-hour recursive forecast.

5.1 Additional Feature Engineering

Beyond the features stored during ingestion, the training pipeline computes additional lag and rolling features specifically for the hourly prediction task:

- AQI lags at 1h, 2h, 3h, 6h, 12h, 24h, and 48h — capturing short and medium-term autocorrelation
- Rolling mean and standard deviation over 6h, 12h, and 24h windows with `min_periods=1`
- `aqi_diff1h`: hour-over-hour change in AQI
- `aqi_diff24h`: change relative to same hour yesterday
- Cyclical encodings of hour and day-of-week using sine/cosine transformations

5.2 Supervised Dataset Construction

Each training sample is constructed as follows: the feature vector at hour t predicts the AQI at hour $t+1$. This single-step formulation takes advantage of the strong autocorrelation in AQI data — adjacent hours are highly correlated — and produces a well-conditioned regression problem. The first 48 rows are skipped to allow lag features to fully populate. The dataset is then split into training and test sets using a time-aware split: the last 20 days (480 hours) are held out as the test set, preserving temporal ordering.

5.3 Model Architecture

All models are wrapped in a scikit-learn Pipeline consisting of three steps: a SimpleImputer with median strategy to handle any residual NaN values, an optional StandardScaler for models that benefit from normalisation, and the estimator itself. Four models are trained and compared:

Model	Key Hyperparameters	Pipeline Steps
Ridge Regression	<code>alpha=10.0</code>	Imputer → StandardScaler → Ridge
Random Forest	<code>n_estimators=300</code> , <code>max_depth=10</code> , <code>min_samples_leaf=3</code>	Imputer → RandomForestRegressor
LightGBM	<code>n_estimators=500</code> , <code>lr=0.05</code> , <code>max_depth=6</code> , <code>num_leaves=31</code>	Imputer → LGBMRegressor
XGBoost	<code>n_estimators=500</code> , <code>lr=0.05</code> , <code>max_depth=6</code> , <code>min_child_weight=3</code>	Imputer → XGBRegressor

5.4 Evaluation

Each model is evaluated on the 20-day held-out test set using three metrics: Root Mean Squared Error (RMSE), Mean Absolute Error (MAE), and R-squared (R^2). The model with the lowest RMSE is selected as the active model for forecasting.

5.5 Model Registry

The winning model's sklearn Pipeline is serialised using Python's pickle module and stored in MongoDB GridFS, which handles binary objects up to 16 MB per chunk and is well-suited for model storage. A metadata document is simultaneously written to a 'model_registry' collection containing the model name, version timestamp, evaluation metrics, feature column list, lookback hours, forecast hours, GridFS file ID, SHA-256 fingerprint, and an `is_active` boolean flag. Previous active models are marked inactive before the new one is inserted, maintaining a full version history.

6. Recursive 72-Hour Forecasting Strategy

The forecasting strategy uses recursive multi-step prediction to generate a 72-hour hourly AQI forecast from a single-step model.

6.1 How Recursive Forecasting Works

Starting from the last known hour in the database, the model predicts AQI one hour ahead. This prediction is then appended to the AQI history and used as a lag feature for the next step. This process repeats 72 times to produce a full 3-day hourly forecast. Weather inputs (temperature, humidity, wind, etc.) are frozen at the last observed values for all future steps, which is a necessary approximation given the absence of a weather forecast feed.

6.2 Daily Aggregation

The 72 hourly predictions are grouped into three 24-hour blocks. Within each block, the mean, minimum, and maximum predicted AQI values are computed. If the model-predicted daily swing (max - min) is smaller than the typical historical daily standard deviation, the min and max are widened symmetrically around the mean to reflect realistic daily variability. This produces three daily forecast records: date, predicted_aqi (daily mean), hourly_min, and hourly_max.

6.3 Forecast Storage

Two MongoDB collections store the forecast output: 'aqi_forecasts' contains 3 daily documents with date, predicted_aqi, hourly_min, hourly_max, generated_at timestamp, and model_used. 'aqi_forecasts_hourly' contains 72 hourly documents with datetime, predicted_aqi_hourly, generated_at, and model_used. Both collections are fully replaced on each pipeline run using delete_many() followed by insert_many().

7. Automation with GitHub Actions

The entire pipeline — data ingestion, model training, and forecast generation — is automated using GitHub Actions and runs once daily at 01:00 UTC (06:00 PKT) without any manual intervention.

7.1 Workflow Structure

The workflow file at `.github/workflows/aqi_forecast.yml` defines a single job with five steps: repository checkout, Python 3.11 environment setup with pip caching, dependency installation from `requirements.txt`, execution of the training pipeline, and finally execution of the feature pipeline to ingest fresh data for the next cycle.

7.2 Secrets Management

Sensitive credentials — `MONGO_URI`, `MONGO_DB`, and `MONGO_COLLECTION` — are stored as GitHub repository secrets and injected as environment variables at runtime. They are never stored in the repository code. The same credentials are independently configured as Streamlit Cloud secrets for the dashboard application.

7.3 Schedule

The cron expression `'0 1 * * *` runs the pipeline at 01:00 UTC daily, corresponding to 06:00 Pakistan Standard Time. This timing is chosen to ensure the previous day's complete archive data is available from Open-Meteo before the pipeline runs. The workflow also supports manual triggering via `workflow_dispatch` for ad-hoc retraining or debugging.

8. Streamlit Dashboard (app.py)

The dashboard is built with Streamlit and deployed on Streamlit Community Cloud, providing a publicly accessible URL. It reads all data directly from MongoDB Atlas and presents it through an interactive, theme-adaptive interface.

8.1 Design System

The dashboard uses a custom CSS design system built on CSS custom properties (variables) that automatically adapt to the user's OS theme preference via a `@media (prefers-color-scheme: dark)` block. Two fonts are imported from Google Fonts: Syne (geometric sans-serif) for headings and numeric displays, and DM Mono (monospace) for body text and data labels. The colour palette uses a dark navy base in dark mode and a clean white base in light mode, with blue accents throughout.

8.2 Dashboard Sections

Section	Content
Top Bar	App name, live timestamp, active model name, RMSE and R^2 score
Current AQI Hero	Large colour-coded AQI number, category label, PM2.5, PM10, Ozone, NO2 pills

Section	Content
Atmospheric Conditions	8 metric cards: temperature, humidity, wind speed, pressure, SO ₂ , CO, wind gusts, cloud cover
3-Day Forecast	Three cards showing daily average AQI, category, daily min and max
72-Hour Hourly Chart	Spline line chart with AQI band fills and day separator lines
14-Day Historical Trend	Hourly AQI line with 24h rolling average overlay
AQI Distribution	Colour-coded histogram of AQI values over the past 14 days
Model Performance	Active model name, RMSE, MAE, R ² , last trained timestamp

8.3 AQI Colour Coding

All AQI values displayed in the dashboard are colour-coded according to the US EPA standard AQI categories:

AQI Range	Colour	Category
0 – 50	#16A34A (Green)	Good
51 – 100	#CA8A04 (Yellow)	Moderate
101 – 150	#EA580C (Orange)	Unhealthy for Sensitive Groups
151 – 200	#DC2626 (Red)	Unhealthy
201 – 300	#9333EA (Purple)	Very Unhealthy
301+	#7F1D1D (Maroon)	Hazardous

8.4 Caching

All MongoDB queries in the dashboard are wrapped in Streamlit's `@st.cache_data` decorator with a TTL of 1800 seconds (30 minutes). This prevents redundant database calls on every user interaction while ensuring the displayed data remains no more than 30 minutes stale. The database connection itself is cached as a resource using `@st.cache_resource`, creating a single shared connection across all sessions.

9. Repository Structure

File / Folder	Purpose
feature_pipeline.py	Data ingestion, cleaning, feature engineering, MongoDB upsert
training_pipeline.py	Model training, evaluation, registry, recursive forecast
app/app.py	Streamlit dashboard application
requirements.txt	Python package dependencies for all components
.github/workflows/aqi_forecast.yml	GitHub Actions cron workflow (daily at 06:00 PKT)
data/	EDA notebooks and exploratory analysis
eda/	Jupyter notebooks for exploratory data analysis
model/	Local model artefact storage (development only)
.gitignore	Excludes .env, __pycache__, local model files

9.1 Dependencies (requirements.txt)

Package	Purpose
numpy, pandas	Numerical computation and data manipulation
scikit-learn	Ridge Regression, Random Forest, Pipeline, Imputer, Scaler, Metrics
lightgbm	LightGBM gradient boosting model
xgboost	XGBoost gradient boosting model
pymongo[srv]	MongoDB Atlas driver with SRV connection string support
requests	HTTP calls to Open-Meteo APIs
python-dotenv	Loads .env credentials for local development
streamlit	Dashboard framework and deployment
plotly	Interactive charting library for the dashboard
pyarrow	Efficient columnar data serialisation

10. Technical Challenges and Resolutions

The development of this project involved numerous technical challenges across environment setup, data engineering, and machine learning. The following documents all major issues encountered and how each was resolved.

10.1 PowerShell Execution Policy Blocking Virtual Environment

Problem: Activating the Python virtual environment via `.venv\Scripts\activate` in PowerShell produced a security error: 'File cannot be loaded because running scripts is disabled on this system.'

Resolution: The execution policy was temporarily relaxed for the current terminal session only, ensuring no permanent system-wide changes:

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process  
.venv\Scripts\activate
```

10.2 Multiple Python Interpreters (3.11 venv vs 3.14 Global)

Problem: Running `pip install` sent packages to the global Python 3.14 installation ('Defaulting to user installation because normal site-packages is not writeable') instead of the project's Python 3.11 virtual environment, causing `ModuleNotFoundError` at runtime.

Resolution: Packages were installed directly using the venv's Python executable to bypass activation ambiguity:

```
D:\AQI_Predictor\.venv\Scripts\python.exe -m pip install -r requirements.txt
```

10.3 IDE (Antigravity) Not Recognising Installed Packages

Problem: Even after correctly installing all packages, the Antigravity IDE threw `ModuleNotFoundError` when running via its built-in run button. Diagnostic testing confirmed `sys.executable` pointed to the correct venv and `site-packages` was in `sys.path`, yet the error persisted.

Root Cause: Antigravity spawned Streamlit as a separate subprocess with a different environment context, isolating it from the venv.

Resolution: The application was launched directly from the Command Prompt through the venv Python:

```
D:\AQI_Predictor\.venv\Scripts\python.exe -m streamlit run app.py
```

10.4 NaN Values Crashing Model Training

Problem: Ridge Regression threw: 'ValueError: Input X contains NaN. Ridge does not accept missing values encoded as NaN natively.'

Resolution: A `SimpleImputer` with median strategy was added as the first step in every model's sklearn Pipeline:

```
steps = [("imputer", SimpleImputer(strategy="median"))]  
if scale: steps.append(("scaler", StandardScaler()))
```

```
steps.append(("model", estimator))
```

10.5 ValueError: Expected 2D Array, Got 1D Array

Problem: During forecast generation, sklearn threw a shape error because the input array was 1D.

Resolution: The forecast input was explicitly reshaped to guarantee a 2D array:

```
last_row = daily[feature_cols].iloc[-1].values.astype(np.float32).reshape(1, -1)
```

10.6 RuntimeError: Not Enough Daily Rows

Problem: A blanket dropna() call after lag feature creation was removing the first 7+ rows (needed for aqi_lag_7d and aqi_roll7_mean), leaving too few rows to build training samples.

Resolution: dropna was changed to only drop rows where the minimum required lag (aqi_lag_1d) was missing, with all other NaN values filled via forward fill and median:

```
daily.dropna(subset=["aqi_lag_1d"], inplace=True)
daily.ffill(inplace=True)
daily[col] = daily[col].fillna(daily[col].median())
```

10.7 Negative R² Scores — Daily Direct Prediction Approach

Problem: All four models produced negative R² scores (e.g. LightGBM R²=-0.617) when predicting daily aggregate AQI 1-3 days ahead from a flattened 30-day window.

Root Cause: Flattening a 30-day window produced ~900 input columns for a dataset with only ~100 samples, causing severe overfitting. Hourly-engineered features lost all meaning after daily averaging.

Resolution: The entire architecture was redesigned to use single-step hourly prediction (predict AQI at t+1 from features at t) with a 48-hour lookback, reducing input dimensionality to ~30 features and leveraging the natural hourly autocorrelation of AQI.

10.8 Recursive Forecast Drift

Problem: The 72-hour recursive forecast climbed monotonically from AQI 73 to 127 with no natural variation, as small positive biases compounded 72 times.

Resolution: Mean reversion dampening was applied at each forecast step, pulling predictions toward the 7-day historical mean proportionally to the forecast horizon:

```
reversion = REVERSION_RATE * step * (hist_mean - raw_pred)
pred_aqi = max(0.0, raw_pred + reversion)
```

10.9 Stale Forecast Dates in Dashboard

Problem: The 3-day forecast displayed dates from a week prior because old MongoDB documents were not refreshed.

Resolution: The training pipeline was re-executed. The `delete_many()` call before each `insert_many()` ensures stale data is always overwritten. GitHub Actions prevents recurrence via daily automated runs.

10.10 Streamlit Cloud Build Failures

Problem: Two successive deployment failures occurred on Streamlit Cloud. First, a typo 'streamlilt' in requirements.txt caused a package not found error. Second, after fixing the typo, plotly was missing from requirements.txt as it had only been installed locally.

Resolution: Both packages were added correctly to requirements.txt and committed to GitHub, triggering an automatic redeploy.

11. Future Improvements

The following improvements are planned for future iterations of the project:

- **Weather forecast integration:** Replace frozen weather inputs in the recursive forecast with actual weather forecast data from Open-Meteo's forecast API, which provides 7-day hourly predictions. This would eliminate a significant source of error in multi-day forecasts.
- **Expanded city coverage:** Extend the platform to monitor AQI in other major Pakistani cities including Lahore, Islamabad, and Peshawar, all of which face severe pollution challenges.
- **Health advisory system:** Implement a rule-based alert system that issues real-time health recommendations based on AQI thresholds, targeted at sensitive groups such as children, elderly, and those with respiratory conditions.
- **SHAP feature importance:** Integrate SHAP (SHapley Additive exPlanations) analysis into the training pipeline to identify which weather and pollution features contribute most to each forecast, improving model interpretability.
- **Deep learning models:** As the dataset grows beyond 6-12 months of hourly data, experiment with LSTM and Transformer architectures which are better suited to long-range sequential dependencies.
- **Model drift detection:** Implement automated monitoring to detect when model performance degrades over time (e.g. due to seasonal distribution shift) and trigger retraining or alerts.
- **API endpoint:** Expose the forecast data through a REST API so other applications and researchers can consume Karachi AQI predictions programmatically.

12. Conclusion

Aab o Hawa demonstrates a complete, production-grade MLOps pipeline applied to a real public health problem. Starting from raw API data, the system automatically ingests, engineers features, trains and evaluates multiple ML models, generates a 72-hour recursive forecast, and serves the results through a publicly accessible interactive dashboard — all on a daily automated schedule with zero manual intervention.

The project encountered and resolved numerous technical challenges spanning environment management, data engineering, and machine learning — each of which contributed to a more robust final system. The recursive forecasting strategy with mean reversion dampening, in particular, represents a practical solution to the drift problem inherent in multi-step autoregressive prediction with limited training data.

As more data accumulates in the MongoDB feature store, model accuracy is expected to improve steadily. The architecture is designed to scale — adding new cities, new data sources, or more sophisticated models requires changes only to the respective pipeline components, leaving the overall system structure intact.